



JERUSALEM COLLEGE OF ENGINEERING
(An Autonomous Institution, Affiliated to Anna University, Chennai)

NOTES

JIT2421-OPERATING SYSTEM

UNIT V- VIRTUAL MACHINES AND MOBILE OS

Virtual Machines - History, Benefits and Features, Building Blocks, Types of Virtual Machines and their Implementations, Virtualization; Mobile OS - iOS and Android Architecture, SDK Frameworks.

VIRTUAL MACHINE:

A virtual machine (VM) is a virtual representation or emulation of a physical computer that uses software instead of hardware to run programs and deploy applications.

HISTORY OF VIRTUAL MACHINES :

1. First appeared in IBM mainframes in 1972
2. Allowed multiple users to share a batch-oriented system
3. Formal definition of virtualization helped move it beyond IBM
 1. A VMM provides an environment for programs that is essentially identical to the original machine
 2. Programs running within that environment show only minor performance decreases
 3. The VMM is in complete control of system resources
4. In late 1990s Intel CPUs fast enough for researchers to try virtualizing on general purpose PCs
 - Xen and VMware created technologies, still used today
 - Virtualization has expanded to many OSes, CPUs, VMMs

Key milestone:

- 1960s: IBM Mainframe Virtualization
- 1999: VMware's first x86 virtualization
- 2006: Launch of KVM (Kernel-based Virtual Machine)
- Present: Hypervisors and container technologies dominate modern virtualization

BENEFITS AND FEATURES OF VM:

1. Host system protected from VMs, VMs protected from each other
 - I.e. A virus less likely to spread
 - Sharing is provided though via shared file system volume, network communication
2. Freeze, suspend, running VM
 - Then can move or copy somewhere else and resume
 - Snapshot of a given state, able to restore back to that state
 - Some VMMs allow multiple snapshots per VM
 - Clone by creating copy and running both original and copy
3. Great for OS research, better system development efficiency
4. Run multiple, different OSes on a single machine

- Consolidation, app dev, ...
5. Templating – create an OS + application VM, provide it to customers, use it to create multiple instances of that combination
 6. Live migration – move a running VM from one host to another!
 - No interruption of user access
 7. All those features taken together -> cloud computing
 - Using APIs, programs tell cloud infrastructure (servers, networking, storage) to create new guests, VMs, virtual desktop.

BUILDING BLOCKS:

Generally difficult to provide an exact duplicate of underlying machine

- Especially if only dual-mode operation available on CPU
- But getting easier over time as CPU features and support for VMM improves
- Most VMMs implement virtual CPU (VCPU) to represent state of CPU per guest as guest believes it to be
 - When guest context switched onto CPU by VMM, information from VCPU loaded and stored

1. Building Blocks - Trap and Emulate:

Dual mode CPU means guest executes in user mode

- Kernel runs in kernel mode
- Not safe to let guest kernel run in kernel mode too
- So VM needs two modes – virtual user mode and virtual kernel mode
 - Both of which run in real user mode
- Actions in guest that usually cause switch to kernel mode must cause switch to virtual kernel mode

How does switch from virtual user mode to virtual kernel mode occur?

- Attempting a privileged instruction in user mode causes an error -> trap
- VMM gains control, analyzes error, executes operation as attempted by guest
- Returns control to guest in user mode
- Known as trap-and-emulate
- Most virtualization products use this at least in part

User mode code in guest runs at same speed as if not a guest

But kernel mode privilege mode code runs slower due to trap-and-emulate

- Especially a problem when multiple guests running, each needing trap-and-emulate

CPUs adding hardware support, mode CPU modes to improve virtualization performance

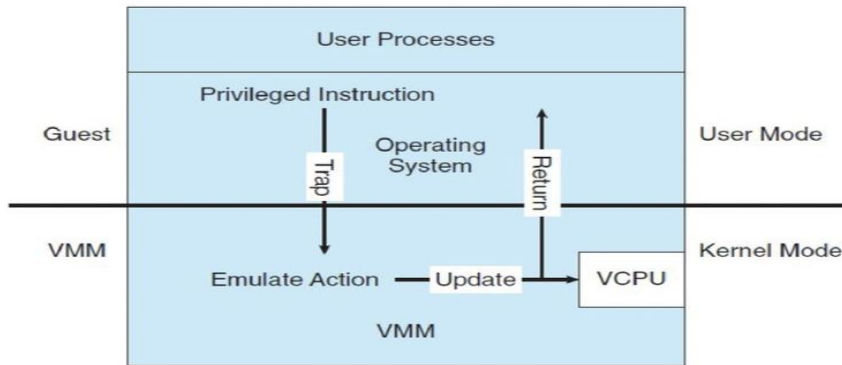


Figure 16.2 Trap-and-emulate virtualization implementation.

2. Building Block - Binary Translation:

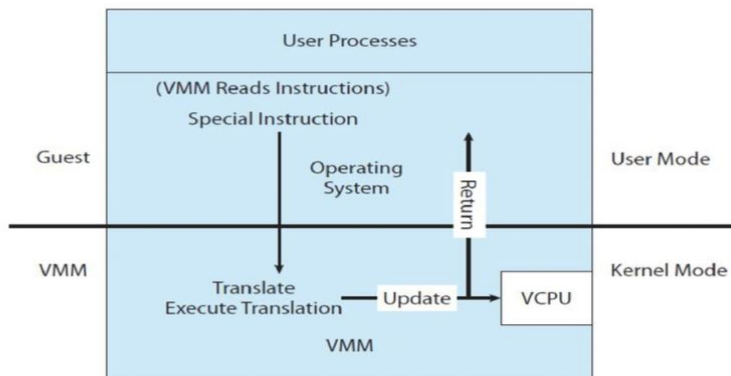


Figure 16.3 Binary translation virtualization implementation.

Some CPUs don't have clean separation between privileged and nonprivileged instructions

- Earlier Intel x86 CPUs are among them
 - Earliest Intel CPU designed for a calculator
- Backward compatibility means difficult to improve
- Consider Intel x86 popf instruction
 - Loads CPU flags register from contents of the stack
 - If CPU in privileged mode -> all flags replaced
 - If CPU in user mode -> on some flags replaced– No trap is generated

Other similar problem instructions we will call special instructions

- Caused trap-and-emulate method considered impossible until 1998

Binary translation solves the problem

- Basics are simple, but implementation very complex
 1. If guest VCPU is in user mode, guest can run instructions natively
 2. If guest VCPU in kernel mode (guest believes it is in kernel mode)
 1. VMM examines every instruction guest is about to execute by reading a few instructions ahead of program counter
 2. Non-special-instructions run natively
 3. Special instructions translated into new set of instructions that perform equivalent task (for example changing the flags in the VCPU)

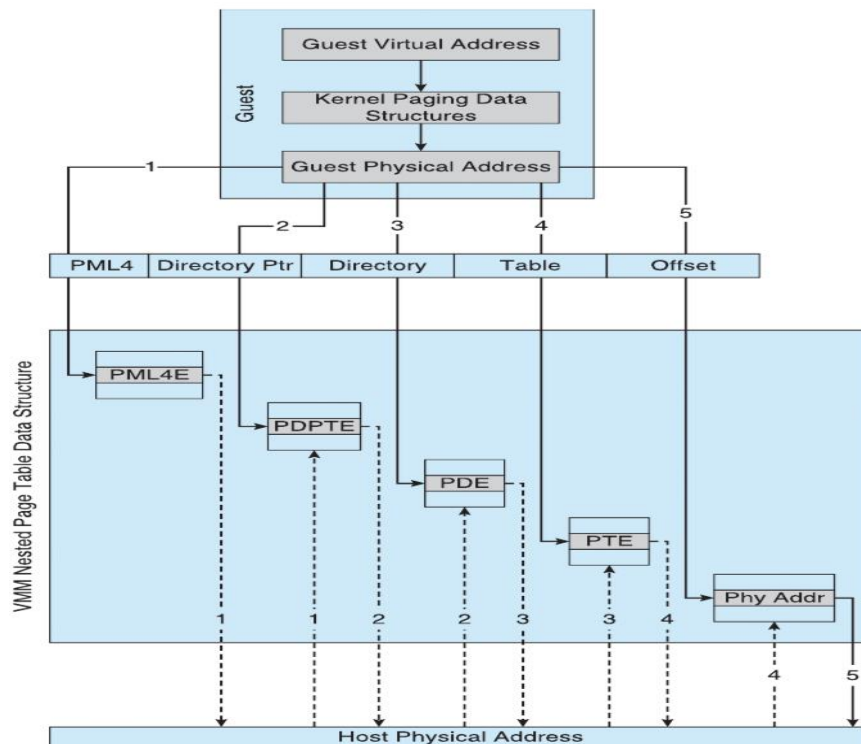
Implemented by translation of code within VMM

Code reads native instructions dynamically from guest, on demand, generates native binary code that executes in place of original code

Performance of this method would be poor without optimizations

- Products like VMware use caching
 - Translate once, and when guest executes code containing special instruction cached translation used instead of translating again
 - Testing showed booting Windows XP as guest caused 950,000 translations, at 3 microseconds each, or 3 second (5 %) slowdown over native

Nested Page Table :



Memory management another general challenge to VMM implementations

How can VMM keep page-table state for both guests believing they control the page tables and VMM that does control the tables

Common method (for trap-and-emulate and binary translation) is nested page tables (NPTs)

Each guest maintains page tables to translate virtual to physical addresses

- VMM maintains per guest NPTs to represent guest's page-table state
 - Just as VCPU stores guest CPU state
- When guest on CPU -> VMM makes that guest's NPTs the active system page tables
- Guest tries to change page table -> VMM makes equivalent change to NPTs and its own page tables
- Can cause many more TLB misses -> much slower performance

Building Blocks Hardware Assistance:

All virtualization needs some HW support

More support -> more feature rich, stable, better performance of guests

Intel added new VT-x instructions in 2005 and AMD the AMD-V instructions in 2006

- CPUs with these instructions remove need for binary translation
- Generally define more CPU modes - "guest" and "host"
- VMM can enable host mode, define characteristics of each guest VM, switch to guest mode and guest(s) on CPU(s)
- In guest mode, guest OS thinks it is running natively, sees devices (as defined by VMM for that guest)
 - Access to virtualized device, priv instructions cause trap to VMM
 - CPU maintains VCPU, context switches it as needed
- HW support for Nested Page Tables, DMA, interrupts as well over time

TYPES OF VIRTUAL MACHINES AND IMPLEMENTATION:

Many variations as well as HW details

- Assume VMMs take advantage of HW features
 - HW features can simplify implementation, improve performance

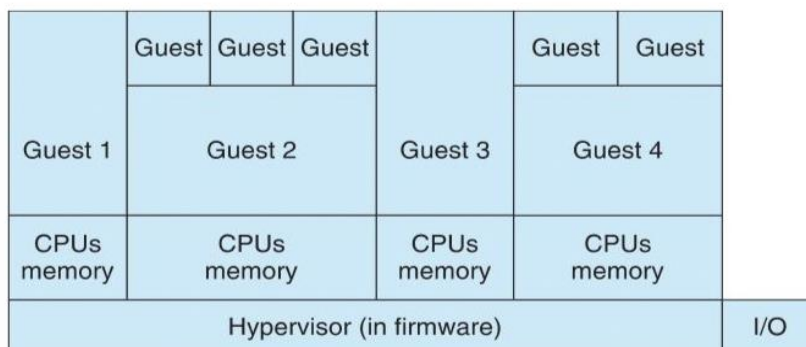
Whatever the type, a VM has a lifecycle

- Created by VMM
- Resources assigned to it (number of cores, amount of memory, networking details, storage details)
- In type 0 hypervisor, resources usually dedicated
- Other types dedicate or share resources, or a mix
- When no longer needed, VM can be deleted, freeing resources

Steps simpler, faster than with a physical machine install

- Can lead to virtual machine sprawl with lots of VMs, history and state difficult to track

Types of VMs -Hypervisor 0:



Old idea, under many names by HW manufacturers

- "partitions", "domains"
- A HW feature implemented by firmware
- OS need to nothing special, VMM is in firmware
- Smaller feature set than other types
- Each guest has dedicated HW

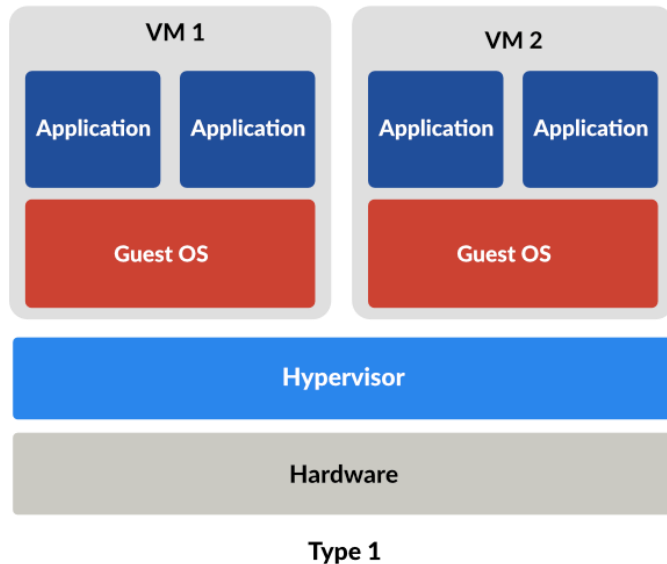
I/O a challenge as difficult to have enough devices, controllers to dedicate to each guest

Sometimes VMM implements a control partition running daemons that other guests communicate with for shared I/O

Can provide virtualization-within-virtualization (guest itself can be a VMM with guests)

- Other types have difficulty doing this

Types of VMs -Hypervisor 1:



Commonly found in company datacenters

- In a sense becoming "datacenter operating systems"
- Datacenter managers control and manage OSes in new, sophisticated ways by controlling the Type 1 hypervisor
- Consolidation of multiple OSes and apps onto less HW
- Move guests between systems to balance performance
- Snapshots and cloning

Special purpose operating systems that run natively on HW

- Rather than providing system call interface, create run and manage guest OSes
- Can run on Type 0 hypervisors but not on other Type 1s

Run in kernel mode

- Guests generally don't know they are running in a VM

Implement device drivers for host HW because no other component can

- Also provide other traditional OS services like CPU and memory management

Another variation is a general purpose OS that also provides VMM functionality

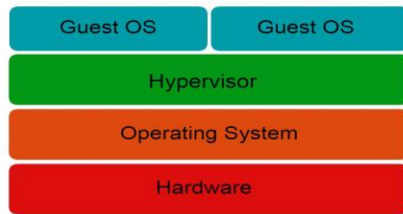
- RedHat Enterprise Linux with KVM, Windows with Hyper-V, Oracle Solaris
- Perform normal duties as well as VMM duties
- Typically less feature rich than dedicated Type 1 hypervisors

In many ways, treat guests OSes as just another process

- Albeit with special handling when guest tries to execute special instructions

Types of VMs - Hypervisor 2:

Type 2 Hypervisor



Less interesting from an OS perspective

- Very little OS involvement in virtualization
- VMM is simply another process, run and managed by host

Even the host doesn't know they are a VMM running guests

- Tend to have poorer overall performance because can't take advantage of some HW features
- But also a benefit because require no changes to host OS

Student could have Type 2 hypervisor on native host, run multiple guests, all on standard host OS such as Windows, Linux, MacOS

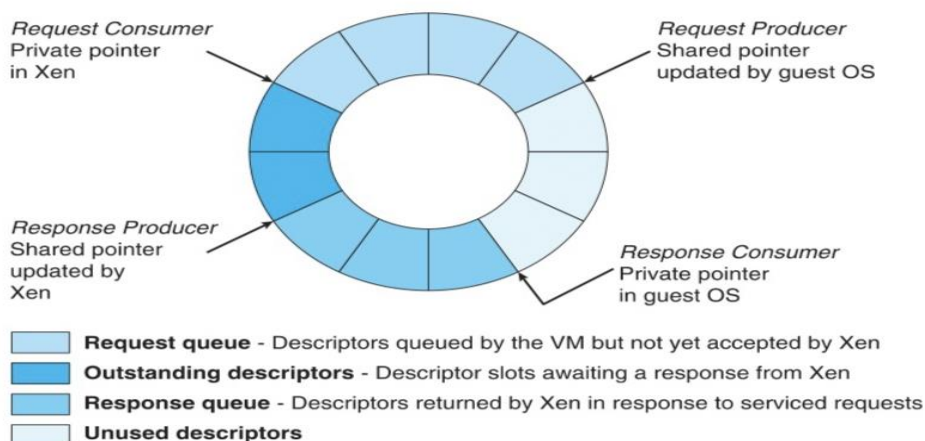
Types of VMs - Paravirtualization:

Does not fit the definition of virtualization - VMM not presenting an exact duplication of underlying hardware

- But still useful!
- VMM provides services that guest must be modified to use
- Leads to increased performance
- Less needed as hardware support for VMs grows

Xen, leader in paravirtualized space, adds several techniques

- For example, clean and simple device abstractions
 - Efficient I/O
 - Good communication between guest and VMM about device I/O
 - Each device has circular buffer shared by guest and VMM via shared memory



Xen, leader in paravirtualized space, adds several techniques

- Memory management does not include nested page tables
 - Each guest has own read-only tables

- Guest uses hypercall (call to hypervisor) when page-table changes needed

Paravirtualization allowed virtualization of older x86 CPUs (and others) without binary translation

Guest had to be modified to use run on paravirtualized VMM

But on modern CPUs Xen no longer requires guest modification -> no longer paravirtualization

Types of VMs - Programming Environment Virtualization:

Also not-really-virtualization but using same techniques, providing similar features

Programming language is designed to run within custom-built virtualized environment

- For example Oracle Java has many features that depend on running in Java Virtual Machine (JVM)

In this case virtualization is defined as providing APIs that define a set of features made available to a language and programs written in that language to provide an improved execution environment

JVM compiled to run on many systems (including some smart phones even)

Programs written in Java run in the JVM no matter the underlying system

Similar to interpreted languages

Types of VMs – Emulation:

Another (older) way for running one operating system on a different operating system

- Virtualization requires underlying CPU to be same as guest was compiled for
- Emulation allows guest to run on different CPU

Necessary to translate all guest instructions from guest CPU to native CPU

Emulation, not virtualization

Useful when host system has one architecture, guest compiled for other architecture

- Company replacing outdated servers with new servers containing different CPU architecture, but still want to run old applications

Performance challenge order of magnitude slower than native code

- New machines faster than older machines so can reduce slowdown

Very popular especially in gaming where old consoles emulated on new

Types of VMs - Application Containment:

Some goals of virtualization are segregation of apps, performance and resource management, easy start, stop, move, and management of them

Can do those things without full-fledged virtualization

- If applications compiled for the host operating system, don't need full virtualization to meet these goals

Oracle containers/zones for example create virtual layer between OS and apps

- Only one kernel running - host OS
- OS and devices are virtualized, providing resources within zone with impression that they are only processes on system

- Each zone has its own applications; networking stack, addresses, and ports; user accounts, etc
- CPU and memory resources divided between zones
- Zone can have its own scheduler to use those resources

MOBILE OS :

Mobile Operating System Structure:

A mobile OS is a software platform on top of which other programs called application programs can run on mobile devices such as PDA, cellular phones, smart phone and etc.



Mobile OS System Structure

Low Level Hardware:

Mobile Processors

Processors use RISC architecture

Mainly two types

1. **ARM Processors:**
 - It is a 32-bit RISC processor architecture
 - Low power consumption
 - Example: Qualcomm Snapdragon, Marvell's XScale & Texas Instruments OMAP series
2. **MIPS-Microprocessor without Interlocked Pipeline Stages:**
 - Mainly used in embedded systems
 - Example: TiVo, Playstation1 & 2

Mobile Memory:

The better the memory management offered by the OS, the wider the options available to applications developers. Mobile devices have two types of memories

1. **ROM-For operating system and preinstalled programs**
2. **RAM-For user information**

Types of RAMS:

1. **DRAM (Dynamic RAM):** cheapest, used in mobile devices
2. **EDO (Enhanced Data Output):** more expensive but offers a speed increase over DRAM(WindowsCE)
3. **SDRAM (Synchronous Dynamic RAM):** a further 50% speed (iPAQ)
4. **DDR (Double Data Rate)** is twice as fast as SDRAM
5. **OUM (Ovonic Unified Memory):** experimental

Kernal:

Responsible for services such as

- Security
- Memory Management

- **Process Management**

It include the following components

- **I/O Components**
- **File Systems**
- **Networking Components**

Libraries & Application Framework:

Libraries:

- **Media Libraries**
- **3D Libraries, etc.**

Application Framework:

- **These Libraries are exposed to developers through the application framework**

Types of Mobile Operating Systems:

When you purchase a mobile device the manufacturer will have chosen the operating system for that specific device Often, you will want to learn about the mobile operating system before you purchase a device to ensure compatibility and support for the mobile applications you want to use.

Popular Mobile Operating Systems:

1. Android OS (Google Inc.):

The Android mobile operating system is Google's open and free software stack that includes an operating system, middleware and also key applications for use on mobile devices, including smartphones. Updates. for the open source Android mobile operating system have been developed under "dessert-inspired" version names (Cupcake, Donut, Eclair, Gingerbread, Honeycomb, Ice Cream Sandwich) with each new version. arriving in alphabetical order with new enhancements and improvements.

2. Bada (Samsung Electronics):

Bada is a proprietary Samsung mobile OS that was first launched in 2010. The Samsung Wave was the first smartphone to use this mobile OS. Bada provides mobile features such as multipoint-touch, 3D graphics and of course, application downloads and installation.

3. BlackBerry OS (Research In Motion):

The BlackBerry OS is a proprietary mobile operating system developed by Research In Motion for use on the company's popular BlackBerry handheld devices. The BlackBerry platform is popular with corporate users as it offers synchronization with Microsoft Exchange, Lotus Domino, Novell Group Wise email and other business software, when used with the BlackBerry Enterprise Server.

4. iPhone OS/iOS (Apple):

Apple's iPhone OS was originally developed for use on its iPhone devices. Now, the mobile operating system is referred to as iOS and is supported on a number of Apple devices including the iPhone, iPad, iPad 2 and iPod Touch. The iOS mobile operating system is available only on Apple's own manufactured devices as the company does not license the OS for third-party hardware. Apple iOS is derived from Apple's Mac OS X operating system.

5. MeeGo OS (Nokia and Intel):

A joint open source mobile operating system which is the result of merging two products based on open source technologies: Maemo (Nokia) and Moblin (Intel) MeeGo is a mobile OS designed to work on a number of devices including smartphones, netbooks, tablets, in-vehicle information systems and various devices using Intel Atom and ARMv7 architectures.

6. Palm OS (Garnet OS):

The Palm OS is a proprietary mobile operating system (PDA operating system) that was originally released in 1996 on the Pilot 1000 handheld. Newer versions of the Palm OS have added support for expansion ports, new processors, external memory cards, improved security and support for ARM processors and smartphones. Palm OS 5 was extended to provide support for a broad range of screen resolutions, wireless connections and enhanced multimedia capabilities and is called Garnet OS.

7. Symbian OS (Nokia):

Symbian is a mobile operating system (OS) targeted at mobile phones that offers a high-level of integration with communication and personal information management (PIM) functionality. Symbian OS combines middleware with wireless communications through an integrated mailbox and the integration of Java and PIM functionality (agenda and contacts). Nokia has made the Symbian platform available under an alternative, open and direct model, to work with some OEMs and the small community of platform development collaborators. Nokia does not maintain Symbian as an open source development project.

8. webOS (Palm/HP):

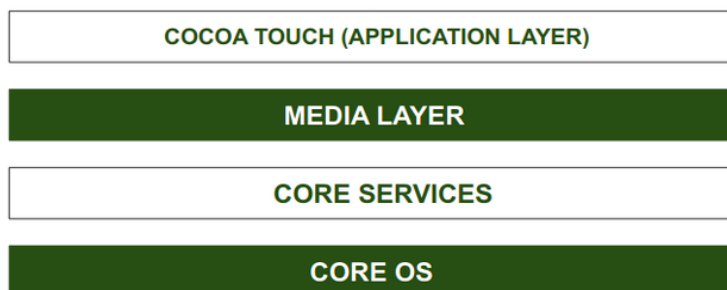
WebOS is a mobile operating system that runs on the Linux kernel. WebOS was initially developed by Palm as the successor to its Palm OS mobile operating system. It is a proprietary Mobile OS which was eventually acquired by HP and now referred to as webOS (lower-case w) in HP literature. HP uses webOS in a number of devices including several smartphones and HP TouchPads. HP has pushed its webOS into the enterprise mobile market by focusing on improving security features and management with the release of webOS 3.x. HP has also announced plans for a version of webOS to run within the Microsoft Windows operating system and to be installed on all HP desktop and notebook computers in 2012.

9. Windows Mobile (Windows Phone):

Windows Mobile is Microsoft's mobile operating system used in smartphones and mobile devices with or without touchscreens. The Mobile OS is based on the Windows CE 5.2 kernel. In 2010 Microsoft announced a new smartphone platform called Windows Phone 7.

iOS ARCHITECTURE:

Architecture of iOS is a layered architecture. At the uppermost level iOS works as an intermediary between the underlying hardware and the apps you make. Apps do not communicate to the underlying hardware directly. Apps talk with the hardware through a collection of well-defined system interfaces. These interfaces make it simple to write apps that work constantly on devices having various hardware abilities. Lower layers give the basic services which all applications rely on and higher-level layer gives sophisticated graphics and interface-related services. Apple provides most of its system interfaces in special packages called frameworks. A framework is a directory that holds a dynamic shared library that is a file, related resources like as header files, images and helper apps required to support that library. Every layer has a set of Framework which the developer uses to construct the applications.



Core OS Layer:

The Core OS layer holds the low-level features that most other technologies are built upon.

- **Core Bluetooth Framework.**
- **Accelerate Framework**
- **External Accessory Framework**
- **Security Services framework**
- **Local Authentication framework.**

64-Bit support from IOS7 supports the 64 bit app development and enables the application to run faster.

Core Services Layer:

Some of the important Frameworks available in the core services layers are detailed

- **Address Book Framework:** Gives programmatic access to a contacts database of user.
- **Cloud Kit Framework:** Gives a medium for moving data between your app and iCloud
- **Core Data Framework:** Technology for managing the data model of a Model View Controller app.
- **Core Foundation Framework:** Interfaces that gives fundamental data management and service features for iOS apps.
- **Core Location Framework:** Gives location and heading information to apps.
- **Core Motion Framework:** Access all motion based data available on a device. Using this core motion framework Accelerometer based information can be accessed.
- **Foundation Framework:** Objective C covering too many of the features found in the Core Foundation framework
- **Healthkit Framework:** New framework for handling health-related information of user
- **Homekit Framework:** New framework for talking with and controlling connected devices in a user's home.
- **Social Framework:** Simple interface for accessing the user's social media accounts.
- **StoreKit Framework:** Gives support for the buying of content and services from inside your iOS. apps, a feature known as In-App Purchase.

Media Layer:

Core Media framework describes the low level interfaces and data types for operating media.

Graphics, Audio and Video technology is enabled using the Media Layer.

Graphics Framework:

- **UIKit Graphics:** It describes high level support for designing images and also used for animating the content of your views.
- **Core Graphics Framework:** It is the native drawing engine for iOS apps and gives support for custom 2D vector and image based rendering
- **Core Animation:** It is an initial technology that optimizes the animation experience of your apps.
- **Core Images,** gives advanced support for controlling video and motionless images in a nondestructive way
- **OpenGL ES and GLKit:** manages advanced 2D and 3D rendering by hardware accelerated interfaces
- **Metal:** It permits very high performance for your sophisticated graphics rendering and computation works. It offers very low overhead access to the A7 GPU.

Audio Framework:

- **Media Player Framework:** It is a high level framework which gives simple use to a user's iTunes library and support for playing playlists.
- **AV Foundation** It is an Objective C interface for handling the recording and playback of audio and video.
- **OpenAL:** is an industry standard technology for providing audio.

Video Framework:

- **AV Kit:** framework gives a collection of easy to use interfaces for presenting video.
- **AV Foundation** gives advanced video playback and recording capability.

Cocoa Touch Layer:

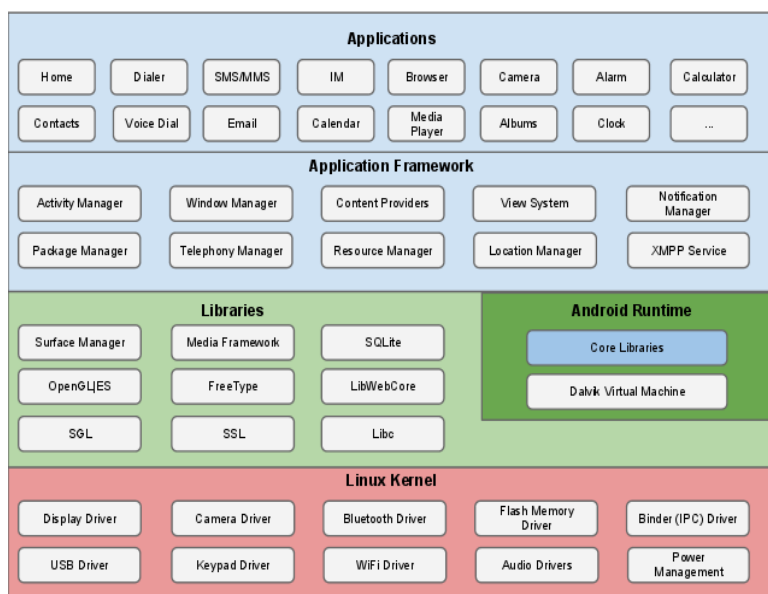
- **UIKit framework:** gives view controllers for showing the standard system interfaces for seeing and altering calendar related events
- **GameKit Framework:** implements support for Game Center which allows users share their game related information online
- **iAd Framework:** allows you deliver banner-based advertisements from your app.
- **MapKit Framework:** gives a scrollable map that you can include into your user interface of app.
- **Push Kit Framework:** provides registration support for VoIP apps.
- **Twitter Framework:** supports a UI for generating tweets and support for creating URLs to access the Twitter service.
- **UIKit Framework:** gives vital infrastructure for applying graphical, event-driven apps in iOS.

Some of the Important functions of UI Kit framework

1. Multitasking support.
2. Basic app management and infrastructure.
3. User interface management
4. Support for Touch and Motion event.
5. Cut, copy and paste support and many more.

ANDROID ARCHITECTURE:

Android operating system is a stack of software components which is roughly divided into five sections and four main layers as shown below in the architecture diagram.



Linux kernel:

At the bottom of the layers is Linux - Linux 3.6 with approximately 115 patches. This provides a level of abstraction between the device hardware and it contains all the essential hardware drivers like camera, keypad, display etc. Also, the kernel handles all the things that Linux is really good at such as networking and a vast array of device drivers, which take the pain out of interfacing to peripheral hardware.

Libraries:

On top of Linux kernel there is a set of libraries including open-source Web browser engine WebKit, well known library libe, SQLite database which is a useful repository for storage and sharing of application data, libraries to play and record audio and video, SSL libraries responsible for Internet security etc.

Android Libraries:

This category encompasses those Java-based libraries that are specific to Android development. Examples of libraries in this category include the application framework libraries in addition to those that facilitate user interface building, graphics drawing and database access. A summary of some key core Android libraries available to the Android developer is as follows:

- **android.app** - Provides access to the application model and is the cornerstone of all Android applications
- **android.content** - Facilitates content access, publishing and messaging between applications and application components
- **android.database** - Used to access data published by content providers and includes SQLite database management classes.
- **android.opengl** - A Java interface to the OpenGL ES 3D graphics rendering API.
- **android.os** - Provides applications with access to standard operating system services including messages, system services and inter-process communication.
- **android.text** - Used to render and manipulate text on a device display.
- **android.view** - The fundamental building blocks of application user interfaces
- **android.widget** - A rich collection of pre-built user interface components such as buttons, labels, list views, layout managers, radio buttons etc.
- **android.webkit** - A set of classes intended to allow web-browsing capabilities to be built into applications. Having covered the Java-based core libraries in the Android runtime, it is now time to turn our attention to the C/C++ based libraries contained in this layer of the Android software stack

Android Runtime:

This is the third section of the architecture and available on the second layer from the bottom. This section provides a key component called Dalvik Virtual Machine which is a kind of Java Virtual Machine specially designed and optimized for Android.

The Dalvik VM makes use of Linux core features like memory management and multi-threading, which is intrinsic in the Java language. The Dalvik VM enables every Android application to run in its own process, with its own instance of the Dalvik virtual machine.

The Android runtime also provides a set of core libraries which enable Android application developers to write Android applications using standard Java programming language.

Application Framework:

The Application Framework layer provides many higher-level services to applications in the form of Java classes. Application developers are allowed to make use of these services in their applications.

The Android framework includes the following key services

- **Activity Manager** - Controls all aspects of the application lifecycle and activity stack.
- **Content Providers**- Allows applications to publish and share data with other applications.
- **Resource Manager** Provides access to non-code embedded resources such as strings, color settings and user interface layouts.
- **Notifications Manager** - Allows applications to display alerts and notifications to the user
- **View System**-An extensible set of views used to create application user interfaces.

Applications

You will find all the Android application at the top layer. You will write your application to be installed on this layer only. Examples of such applications are Contacts Books, Browser, Games etc.

SDK FRAMEWORK:

SDK Stands for "Software Development Kit." An SDK is a collection of software used for developing applications for a specific device or operating system. Examples of SDKs include the Windows 7 SDK, the Mac OS X SDK, and the iPhone SDK.

SDKs typically include an integrated development environment (IDE), which serves as the central programming interface. The IDE may include a programming window for writing source code, a debugger for fixing program errors, and a visual editor, which allows developers to create and edit the program's graphical user interface (GUI). IDEs also include a compiler, which is used to create applications from source code files.

Most SDKs contain sample code, which provides developers with example programs and libraries. These samples help developers learn how to build basic programs with the SDK, which enables them to eventually. create more complex applications. SDKs also offer technical documentation, which may include tutorials and FAQs. Some SDKs may also include sample graphics, such as buttons and icons, which can be incorporated into applications.

Since most companies want to encourage developers to create applications for their platform, SDKs are usually provided free of charge. Developers can simply download an SDK from a company's website and begin programming immediately. However, since each software development kit is different, it can take awhile for developers to learn how to use a new SDK Therefore, most modern SDKs include extensive documentation and have an intuitive programming interface, which helps incentivize program development.

IOS SDK:

The iOS SDK (Software Development Kit) (formerly iPhone SDK) is a software development kit developed by Apple Inc. The kit allows for the development of mobile apps on Apple's iOS operating system.

The SDK is a free download for users of Mac personal computers. It is not available for Microsoft Windows PCs. The SDK contains sets giving developers access to various functions and services of iOS devices, such as hardware and software attributes. It also contains an iPhone simulator to mimic the look and feel of the device on the computer while developing. New versions of the SDK accompany new versions of iOS. In order to test applications, get technical support, and distribute apps through App Store, developers are required to subscribe to the Apple Developer Program

Combined with Xcode, the iOS SDK helps developers write iOS apps using officially supported programming languages, including Swift and Objective-C Other companies have also created tools that allow for the development of native 1OS apps using their respective programming languages.

iOS SDK Frameworks:

The following lists the frameworks that are part of the iOS SDK as of version 9.1.

Accelerate, Accelerate.vimage, Accelerate.vecLib, Accounts, AdSupport (new in 6.0), Address Book, Address BookUI, Assets Library, AudioToolbox, AudioUnit, AVFoundation, AVKit (new for iOS in 8.0), CFNetwork, CloudKit (new in 8.0), Contacts (new in 9.0), Contacts UI (new in 9.0), Core Audio, CoreAudioKit (new for iOS in 8.0), Core Bluetooth, Core Data, CoreFoundation-lower-level C System APIs, CoreGraphics, CoreImage, CoreLocation, Core MIDI, Core Media, bCore Motion, CoreSpotlight (new in 9.0), Core Telephony, Core Text, Core Video, EventKit, EventKitUI, ExternalAccessory, Foundation Base library of the standard NS" classes shared with all Cocoa SDKs, GLKit, GameKit, GameController (new in 7.0), GameplayKit (new in 9.0), HealthKit (new in 8.0), HomeKit (new in 8.0), iAd, ImageIO, JavaScriptCore (new for iOS in 7.0), Local Authentication (new in 8.0), MapKit, MediaPlayer, MediaAccessibility (new in 7.0), MediaToolbox (new in 6.0), Message UI, Metal (new in 8.0), MetalKit (new in 9.0), MetalPerformanceShaders (new in 9.0), MobileCore Services, ModelIO (new in 9.0), Multipeer Connectivity (new in 7.0), NetworkExtension (new in 8.0), NewsstandKit, NotificationCenter (new in 8.0), OpenAL, OpenGL ES, PassKit (new in 6.0), Photos (new in 8.0), PhotosUI (new in 8.0), PushKit (new in 8.0), Quartz Core, QuickLook, ReplayKit (new in 9.0), SafariServices (new in 7.0), SceneKit (new for iOS in 10.8), Security-Keychain access and other security-related APIs, Social (new in 6.0) Integration with Twitter, Facebook. Sina Weibo and other social networks, Sprite Kit (new in 7.0), Store Kit, SystemConfiguration, Twitter - Integration with Twitter. Deprecated, you should use the Social framework instead, UIKit-The core framework for creating iOS, watchOS and tvOS user interfaces, Video Toolbox (new for iOS in 8.0), Watch Connectivity (new in 9.0) Communicate with watchOS 2.0 apps, WatchKit (new in 8.2)-Build watchOS 1.0 apps (not actively supported in Elements), WebKit (new for iOS in 8.0), ril-The base C run-time library (/usr/include) and Elements' own base types, libTofeeHelper types for Elements compiler features, libSwift-Helper types for the Swift language.

Example:

1. UIKit:

Purpose:

Core framework for building user interfaces on iOS.

Handles views, controls, navigation, gestures, and animations.

Common Uses:

Creating screens, buttons, labels, and user input forms.

Example: Displaying a Simple Alert

```
import UIKit
```

```
class ViewController: UIViewController {
```

```
    override func viewDidLoad() {
```

```
        super.viewDidLoad()
```

```
        showAlert()
```

```
    }
```

```
    func showAlert() {
```

```
        let alert = UIAlertController(title: "Hello", message: "Welcome to iOS!", preferredStyle: .alert)
```

```
        alert.addAction(UIAlertAction(title: "OK", style: .default))
```

```
        present(alert, animated: true)
```

```
    }
```

```
}
```

2. Foundation:

Purpose:

Provides essential data types (e.g., String, Array), date handling, file management, and more.

Common Uses:

Working with dates, JSON data, and background tasks.

Example: Formatting a Date

```
import Foundation
let date = Date()
let formatter = DateFormatter()
formatter.dateFormat = "yyyy-MM-dd HH:mm:ss"
print("Current Date:
```

3. EventKit Framework:

Purpose:

Access and manage calendar events and reminders.

Example: Add an Event to Calendar

```
import EventKit
let eventStore = EKEventStore()
func addEvent() {
    eventStore.requestAccess(to: .event) { granted, error in
        if granted {
            let event = EKEvent(eventStore: eventStore)
            event.title = "Meeting"
            event.startDate = Date()
            event.endDate = Date().addingTimeInterval(3600) // 1 hour later
            event.calendar = eventStore.defaultCalendarForNewEvents
            do {
                try eventStore.save(event, span: .thisEvent)
                print("Event added successfully!")
            } catch {
                print("Error saving event:
```

4. Contacts Framework:

Purpose:

Access and manage the user's contacts.

Example: Fetching Contacts List

```
import Contacts
func fetchContacts() {
    let store = CNContactStore()
    store.requestAccess(for: .contacts) { granted, error in
        if granted {
            let keys = [CNContactGivenNameKey, CNContactFamilyNameKey]
            let request = CNContactFetchRequest(keysToFetch: keys as [CNKeyDescriptor])
            do {
                try store.enumerateContacts(with: request) { contact, _ in
                    print("Name:
```